

Recopilacio examens finals.pdf



Lush_8



Gráficos



3º Grado en Ingeniería Informática



Facultad de Informática de Barcelona (Fib)
Universidad Politécnica de Catalunya



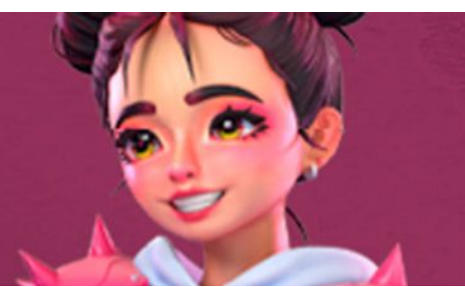
[Accede al documento original](#)



TECNOLOGÍA
a PRECIO de
ESTUDIANTE
VERANO como ESTRELLA
DEL ROCK



webuy.com
CLIC AQUÍ



PIDE INFO Y
RESERVA TU
PLAZA

El punt 3D que resulta d'aplicar la transformació representada per la matriu al punt és:

Agafar la matriu i multiplicar-la pel punt, dividir entre W i ens quedem amb els 3 primers valors:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Punt (6, 18, 12, 3) resultat (6, 18, 36, 3) -> (2, 6, 12)

Si la matriu és de translació: agafes el punt (x,y,z,w), ho divideixes tot entre w -> (x/w,y/w,z/w) i li sumes translació en la X, Y, i Z.

Siguin

P = punt visible de l'escena en una certa direcció de visió **w**

N = normal de la superfície en el punt **P**

L = vector unitari del punt **P** cap a la llum

En última instància, resoldre el problema de la il·luminació global, a nivell de cada punt de la imatge, equival a estimar...

Trieu-ne una:

- ☒ Lo(P, -w) ✓
- ☐ La irradiància de P en direcció w
- ☐ La intensitat en P
- ☐ La irradiància que arriba a P

A vegades no surt Lo(P, -w) sinó "La radiància de sortida al punt P en direcció -w"

En el context de la renderització i la il·luminació global (basada en l'Equació de Renderització de Kajiya), l'objectiu final per determinar el color d'un píxel és calcular la Radiància sortint (Lo).

En la formulació de normal mapping que hem vist a classe, $\left(\frac{\partial F}{\partial u}, \frac{\partial F}{\partial v} \right)$ és...

El gradient del normal map

- ☒ El gradient del bump map
- ☐ Un vector tangent a la superfície, en espai tangent
- ☐ Les components (x,y) d'un vector perpendicular a la superfície

Indica la matriu que s'utilitza a la tècnica de shadow mapping per obtenir les coordenades de textura (s,t,p,q) d'un vèrtex, si el vèrtex es troba en object space.
T(0.5)*S(0.5)*P*V*M

Depenent del dia, el zoom d'un projector d'ajusta per projectar sobre un pantalla A de 2 x 1 m, o sobre una pantalla B de 4 x 2 m. Indica la resposta correcta:

- ☐ La irradiància és la mateixa a les dues pantalles
- ☐ La intensitat lluminosa a la pantalla B és el 25% que la intensitat lluminosa a la pantalla A
- ☐ El flux radiant a la pantalla A és el 25% del flux radiant a la pantalla B
- ☒ La irradiància a la pantalla B és el 25% de la irradiància a la pantalla A

“Els lux mesurats a la pantalla B són el 25% dels mesurats a la pantalla A” també és correcte.

$$E = \frac{\Phi}{A}$$

E és irradiància, Phi és flux radiant (projector), A és àrea

A la parametrització equirectangular estudiada a classe, el punt amb coordenades esfèriques (en radians) theta = 1.5, phi = -0.6 correspon (aproximadament) al punt de l'esfera...

- ☐ (0.06, -0.56, 0.82)
- ☒ (0.82, -0.56, 0.06)
- ☐ (0.18, -0.56, 0.06)
- ☐ (-0.56, 0.82, 0.06)

$r_{xz} = \cos(\phi)$ $Y = \sin(\phi)$ $Z = r_{xz} \cdot \cos(\theta)$
 $X = r_{xz} \cdot \sin(\theta)$

En la versió de parallax mapping que hem vist a classe, el desplaçament que s'aplica a les coordenades de textura, és directament proporcional a...

- ☐ El gradient del height field a (s,t)
- ☐ El gradient del normal map a (s,t)
- ☐ El valor del normal map a (s,t)
- ☒ El valor del height field a (s,t)

Indica el valor que ha de tenir la constant S en aquest fragment de codi:

```
const unsigned int S = ____;  
GLubyte read[S];  
glReadPixels(x, y, 2, 3, GL_RGBA, GL_UNSIGNED_BYTE, read);
```

S = 24

Volem 2*3 = 6 píxels, format GL_RGBA té 4 components, GL_UNSIGNED_BYTE cada component ocupa 1 byte. Total = 6 píxels*4 bytes/píxel = 24 bytes

En quin espai no és correcte interpolar simplement (s,t) quan usem una càmera perspectiva?

- ☒ NDC ✓
- ☐ eye space
- ☐ clip space
- ☐ world space

window space també és una resposta correcta

Considera un sistema de cinema 360° basat en una pantalla gegant de forma esfèrica, de radi $R=22$ m, la qual rep imatges projectades per un projector 360° situat al centre. Si el projector emet 2500 lumens, distribuïts de manera uniforme en totes direccions, indica la il·luminància resultant en la pantalla.

$$E = \Phi / \text{Area} \quad \Phi = 2500 \quad \text{Area (esfera)} = 4 \cdot \pi \cdot R^2 = 1936 \cdot \pi \quad E = 0.411$$

Per un determinat pixel (x,y), els valors al frame buffer són: $\text{depthBuffer}[x,y]=0.5$, $\text{stencilBuffer}[x,y]=4$. El test s'ha configurat amb $\text{glStencilTest}(\text{GL_ALWAYS}, 6, 255)$. Si es genera un fragment per aquest pixel, amb $\text{gl_FragCoord.xyz} = (x, y, 0.6)$, indica quin serà el resultat final al stencilBuffer, si l'operació està configurada amb: $\text{glStencilOp}(\text{GL_DECR}, \text{GL_ZERO}, \text{GL_REPLACE})$;

- ☐ 6
- ☒ 0 ✓
- ☐ 5
- ☐ 4

GL_always, stencilTest sempre passa. Valor entrant $Z_{\text{new}} = 0.6$, $Z_{\text{old}} = 0.5$, per defecte fem GL_LESS que comprova si $\text{new} < \text{old}$, però $0.6 > 0.5$, depth test falla. $\text{glStencilOp}(\text{stencilFail}, \text{depthFail}, \text{depthPass})$, executem GL_ZERO. (Si depth test hagués passat s'hauria executat GL_REPLACE i el valor seria 6)

Tria l'espai de coordenades en que ha d'estar P per tal que la transformació $\text{modelViewProjectionMatrix} \cdot P$ tingui sentit

- ☒ object space ✓
- ☐ eye space
- ☐ world space
- ☐ clip space

Object -> World -> Eye -> Clip -> NDC -> Window/Screen

Una superfície plana perfectament difosa rep una irradiància de 4 W/m^2 . Si L és la radiància reflectida de sortida en direcció perpendicular a la superfície, i L' és la radiància reflectida de sortida a 10 graus de la normal, llavors...

- ☐ $L' = L / \cos(10)$
- ☐ $L' = L / \cos(80)$
- ☒ $L = L'$ ✓
- ☐ $L' = L * \cos(10)$

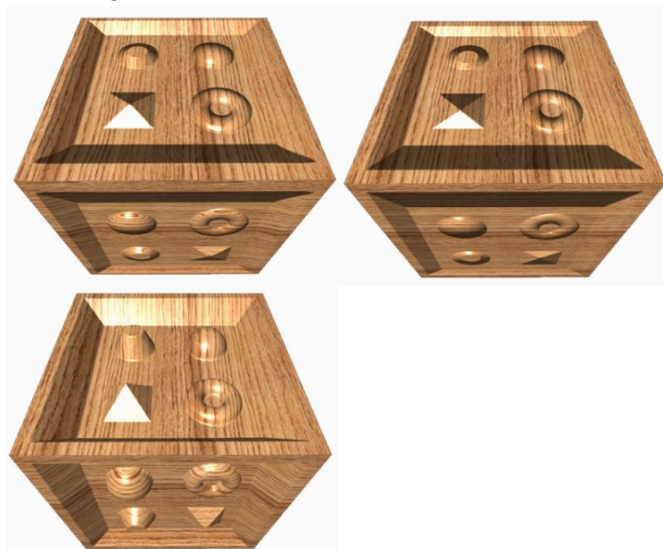
Superfície Lambertiana té radiància constant en totes les direccions

Indica el valor que ha de tenir la variable entera E en aquest fragment de codi, si estem dibuixant una malla de triangles amb 29 cares, 43 arestes i 13 vèrtexs:

```
g.glBindVertexArray(VAO);
g.glDrawElements(GL_TRIANGLES, E, GL_UNSIGNED_INT, (GLvoid*)0);
```

$E = \text{cares} * 3 = 29 * 3$ Utilitzem $GL_TRIANGLES$ (3 índex per triangle)

Indica, d'esquerra a dreta, quines són les tècniques de texturat que s'han aplicat a cada imatge:



- ☐ Relief mapping, Parallax mapping, Normal mapping
- ☐ Displacement mapping, Normal mapping, Relief mapping
- ☐ Normal mapping, Parallax mapping, Relief mapping
- ☒ Parallax mapping, Normal mapping, Relief mapping ✓

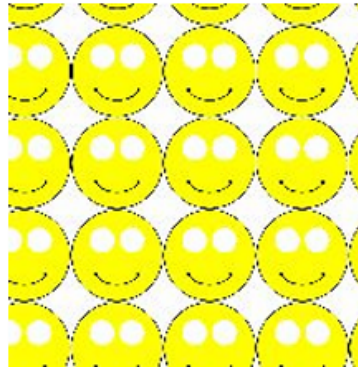
Normal mapping és la més "plana", no té profunditat real, semblen adhesius.
Parallax Mapping, es veu profunditat, és una millora respecte NormalMapping
Relief Mapping, és la més avançada, efecte profunditat real.
Displacement Mapping, les vores rectes del cub estarien deformades pels forats.

Indica el valor de X per tal que el FS generi aquesta sortida amb l'objecte plane i la textura smile.

```
uniform sampler2D smile;

const float X = ???;

void main()
{
    vec2 p = vtexCoord;
    vec2 st = p * X + vec2(1/X);
    fragColor = texture(smile, st);
}
```



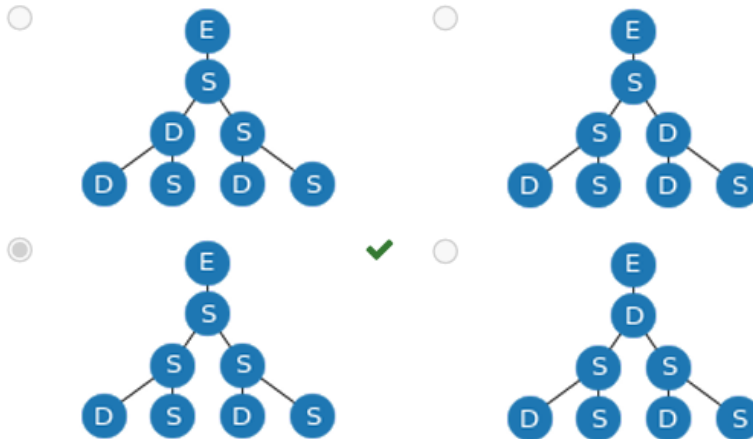
X = 4

p es mou entre 0 i 1, multiplicant per X el nou rang serà de 0 a X. Quan les coordenades superen 1 amb GL_REPEAT, la imatge es repeteix X vegades (podem veure 4 cares amb un cert offset).

Si coneixem l'àrea del conjunt de fulles d'una planta, quina de les magnituds de radiometria o de fotometria estudiades a classe ens permetria calcular millor la quantitat d'energia captada durant la fotosíntesi?

- ☒ Irradiància ✓
- ☐ Luminància
- ☐ Flux lluminós
- ☐ Intensitat radiant

Indica quin arbre de rajos pot ser generat per Ray Tracing clàssic:



S specular, raig rebota o travessa, poden tenir fills.

D diffuse, (Ray Tracing clàssic) calcula il·luminació local i s'atura, no pot tenir fills.



MUSIC
SCHOOL

CONVIERTE TU PASIÓN POR LA MÚSICA EN UNA CARRERA PROFESIONAL

Domina la **industria musical** y
lanza tu carrera con expertos.

ENCUENTRA
— TU SONIDO



MIRA AQUÍ

Assigna a cada crida/ tasca l'ordre relatiu (1,2,3,4) en que s'executa en un pipeline d'OpenGL sense GS:

glGenVertexArrays	1	✓
Call to glDrawArrays	2	✓
Perspective division	3	✓
Alpha blending	4	✓

glGenVertexArrays → 1, Call to glDrawArrays → 2,
Perspective division → 3, Alpha blending → 4.

Indica quina és l'opció més adient per a completar aquest codi a l'espai o espais indicats per '_____':

// draw a scene containing opaque and semitransparent objects

void X:paintGL()

```
{  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);  
    glEnable(GL_DEPTH_TEST);  
    glDepthMask(GL_TRUE);  
    opaque_objects.draw(); // unsorted  
    glEnable(GL_BLEND);  
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);  
    _____;  
    semitransparent_objects.draw(); // unsorted  
    glDisable(GL_BLEND);  
}
```

- ☐ glBlendComb(GL_SUB)
- ☐ glDepthMask(GL_TRUE)
- ☒ glDepthMask(GL_FALSE) ✓
- ☐ glDisable(GL_BLEND)

Dispossem d'aquesta textura:

G H I J K L M N O P

Indica amb quina opció el FS de sota obté aquest resultat amb l'objecte plane:

N

Recorda que plane.obj té coordenades de textura en [0,1].

fragColor = texture(colorMap, factor*vtexCoord + offset)

factor=vec2(0.1, 1.0); offset=vec2(0.7, 0.0);

Tenim 10 lletres en eix X, 1/10. Volem N->0.7 (començant per G com 0, H com 0.1, ...)



Les dades que li arriben interpolades al FS per cada fragment corresponen a...

- ☐ Variables uniform
- ☐ Variables layout del VS
- ☒ Sortides del VS
- ☐ No vull contestar la pregunta
- ☐ Dades dels VBOs

$$\begin{bmatrix} 1 & 0 & 0 & 5 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

La matriu representa un/una...

- ☐ No vull contestar la pregunta
- ☐ Projecció
- ☒ Translació
- ☐ Rotació
- ☐ Escalat

Donat el punt (1.00, 3.00, 6.00), una representació equivalent en coordenades homogènies és...

- ☐ (4.00, 14.00, 24.00, 4.00)
- ☐ No vull contestar la pregunta
- ☐ (2.00, 6.00, 24.00, 2.00)
- ☒ (4.00, 12.00, 24.00, 4.00)
- ☐ (1.00, 3.00, 6.00, 0.00)

$$w = 4, (x*w, y*w, z*w, w)$$

Per a un determinat fragment, les derivades parcials de les coordenades de textura,

un cop multiplicades produeixen aquests valors: $\frac{\partial u}{\partial x} = \frac{\partial v}{\partial y} = 64$; $\frac{\partial u}{\partial y} = \frac{\partial v}{\partial x} = 0$

Quin és el LoD més adient per a accedir a textura:

LoD (level of detail) = $\log(p) = \log(64) = 6$

$$\rho = \max \left(\sqrt{\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2}, \sqrt{\left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2} \right)$$



Tria l'espai de coordenades en que ha d'estar P per tal que la transformació $\text{projectionMatrix} * P$ tingui sentit:

- ☒ eye space
- ☐ No vull contestar la pregunta
- ☐ object space
- ☐ world space
- ☐ clip space

$$\text{ProjectionMatrix} * \text{EyeSpace} = \text{ClipSpace}$$

L'expressió GLSL que representa l'expressió matemàtica $K_d I_d (N \cdot L)$ és:

- ☐ No vull contestar la pregunta
- ☐ $\text{matDiffuse} * \text{lightDiffuse} * N * L$
- ☒ $\text{dot}(N, L) * \text{matDiffuse} * \text{lightDiffuse}$
- ☐ $\text{matDiffuse} * \text{lightDiffuse} * \text{cross}(N, L)$
- ☐ $\text{matDiffuse} * \text{lightDiffuse} * \text{normalize}(N) * L$

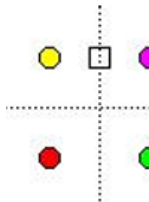
K del material, I de la llum

d: diffuse, s: specular, a: ambient

Tria l'espai de coordenades en que ha d'estar P per tal que la transformació $\text{modelMatrixInverse} * P$ tingui sentit: world space

La figura representa un grup de 2x2 texels, amb diferents colors RGB (interior de cada cercle): Una mostra bilinial al quadrat retornarà aproximadament el color RGB...

- ☒ (1.00, 0.50, 0.50)
- ☐ (0.50, 0.50, 1.00)
- ☐ No vull contestar la pregunta
- ☐ (0.33, 0.50, 0.50)
- ☐ (1.00, 0.25, 0.50)



Quadrat en mig de groc(1, 1, 0) i magenta(1, 0, 1)
Mitjana(1, 0.5, 0.5)

Selecciona la única matriu de projecció (projectionMatrix) plausible per a una càmera perspectiva:

Consultar Projection Matrix (Proyección)

Si volem crear una piràmide de mipmapping completa a partir d'una textura de 32x32 texels, quants nivells calen:

Fer meitats fins a 1x1. En total hi ha 6 nivells. Fórmula: $\log(n) + 1$

32 (nivell 0) -> 16 (nivell 1) -> 8 (nivell 2) -> 4 (nivell 3) -> 2 (nivell 4) -> 1 (nivell 5)



Disposen d'aquesta textura:

Volem texturar un polígon rectangular situat sobre el pla $Z = 0$. Sabem que el seu vèrtex mínim té coordenades (6, 2, 0). Si usem dos plans (S,T) per a generar les coordenades de textura, indica l'opció que (ignora la relació d'aspecte):



$s = Ax + D$ $t = By + D$ $D = 0$ $x = 6, y = 2$
 en horitzontal 5 cops, vertical 3 cops.
 $s = 5$ $5 = A \cdot 6$ $A = 5/6$ $S = \text{vec4}(0.833, 0, 0, 0)$
 $t = 3$ $3 = B \cdot 2$ $B = 3/2$ $T = \text{vec4}(0, 1.5, 0, 0)$

Si ens donen el punt mínim i màxim podem fer:

Volem texturar un polígon rectangular situat sobre el pla $Z = 0$. Sabem que el seu vèrtex mínim té coordenades (0,0,0), i el vèrtex màxim té coordenades (4, 3, 0). Si usem dos plans (S,T) per a generar les coordenades de textura, indica l'opció que permet texturar el polígon així (ignora la relació d'aspecte):

3 cops horitzontal, 4 en vertical
 factor = repeticions/mida

$s = 3/4$ $t = 4/3$ $s = \text{vec4}(0.75, 0, 0, 0)$ $t = \text{vec4}(0, 1.33, 0, 0)$

Indica quina funció GLSL ens permet projectar els vèrtexs d'un objecte sobre una esfera unitària centrada

- ☐ project
- ☐ reflect
- ☐ No vull contestar la pregunta
- ☒ normalize
- ☐ projectSphere



Les coordenades de textura (s,t) que rep un VS en general seràn dins l'interval (tria l'opció correcta més representativa)

- ☐ No vull contestar la pregunta
- ☐ [-1,1]
- ☐ (0,1)
- ☐ (-1, 1)
- ☒ $[-\infty, \infty]$

Indica el tipus de la següent expressió (en el context dels shaders del laboratori):
0.5*vtexCoord: vec2

Les diferents etapes del pipeline d'OpenGL (VS, etc) comencen a executar-se quan s'invoca la funció...

glDrawArrays() o glDrawElements()

Siguin:

M: submatriu 3x3 de la modelMatrix

V: submatriu 3x3 de la viewMatrix,
la normalMatrix es pot calcular com...

- ☐ $(MV)^{-1}$
- ☐ V^{-T}
- ☒ $(VM)^{-T}$
- ☐ $(MV)^{-T}$
- ☐ No vull contestar la pregunta

Indica la transformació geomètrica que no es pot aplicar com el producte d'una matriu 3x3 per un punt (x, y, z):

projecció i translació (qualsevol dels dos és correcte)

El punt amb coordenades homogènies (6.00, 18.00, 6.00, 2.00) correspon al punt 3D...
(6/2, 18/2, 6/2) -> (3, 9, 3)

Indica en quina d'aquestes etapes del pipeline cal interpolar les sortides (variables out) del VS:

- ☐ No vull contestar la pregunta
- ☐ Back face culling
- ☒ Clipping
- ☐ Viewport transformation
- ☐ Texture filtering



Per un determinat pixel (x,y), els valors al frame buffer són: $\text{depthBuffer}[x,y]=0.5$, $\text{stencilBuffer}[x,y]=4$. El test s'ha configurat amb $\text{glStencilTest}(\text{GL_ALWAYS}, 6, 255)$. Si es genera un fragment per aquest pixel, amb $\text{gl_FragCoord.xyz} = (x, y, 0.6)$, indica quin serà el resultat final al stencilBuffer, si l'operació està configurada amb: $\text{glStencilOp}(\text{GL_ZERO}, \text{GL_INCR}, \text{GL_REPLACE})$

Stencil sempre passa, $Z_{\text{new}} = 0.6$, $Z_{\text{old}} = 0.5$, Depth per defecte GL_LESS ($\text{new} < \text{old}$), això evalua fals, Stencil passa però Depth falla, per tant executem GL_INCR , $4 + 1 = 5$ (En totes les preguntes d'aquest tipus s'executa el segon argument de glStencilOp)

Tenim una escena tancada que conté 66 objectes difosos i 5 llums puntuals. Volem generar una imatge de 1024×768 pixels amb Ray Tracing clàssic. Quants shadow rays cal llançar? (indica la resposta amb un enter)

Només obj difosos, sense anti-aliasing, fórmula:

$N_{\text{raigs}} = \text{Width} * \text{Height} * \text{llums_puntuals} = 1024 * 768 * 5 = 3932160$

Assigna a cada crida/tasca l'ordre relatiu (1,2,3,4) en que s'executa en un pipeline d'OpenGL sense GS:

$\text{glBufferData} \rightarrow \text{VS execution starts} \rightarrow \text{Clipping to viewing frustum} \rightarrow \text{Depth test}$

Indica, en un FS, quina transformació de la z en window space té un efecte equivalent a invertir el depth test amb $\text{glDepthFunc}(\text{GL_GREATER})$:

$\text{gl_FragDepth} = 1 - \text{gl_FragCoord.z}$;

La matriu que representa una reflexió respecte un mirall triangular definit pels vèrtexs (6,0,9), (9,0,1), (7,0,1) és...

Observem que té $y = 0$ en tots, per tant està en el pla XZ i reflexió en Y, $\text{scale}(1, -1, 1)$

Assigna a cada tasca l'ordre relatiu en que s'executa, per simular reflexions especulars utilitzant la tècnica de sphere mapping en eye space:

- 1 El VS passa P, N a eye space
- 2 Es calcula el vector reflectit
- 3 Es calculen les coordenades (s,t) del fragment
- 4 Es pren una mostra de la textura que conté el sphere map

Quin terme o factor que apareix a l'equació general del rendering és el que retorna, de forma molt aproximada, un shader que implementa il·luminació de Phong?

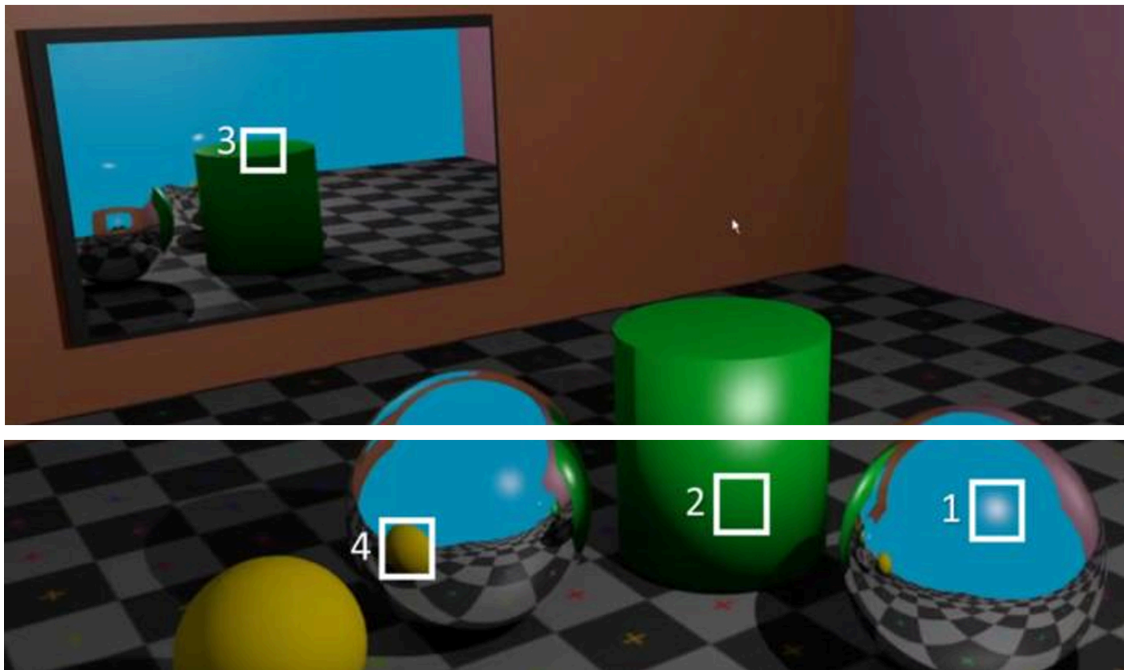
- ☐ $L_i(x, w_i, t)$
- ☒ $L_o(x, w_o, t)$
- ☐ $L_e(x, w_o, t)$
- ☐ No vull contestar la pregunta
- ☐ $f_r(x, w_i, w_o, t)$

L_i és la llum incident

L_o és resultat final (color píxel)

f_r és la BRDF

El light path que explica el color dominant al pixel central del quadrat 3 és...



LDSE

els camins es llegeixen desde llum (L) fins als ulls (E). Fem camí invers.

Raig surt dels ulls, el mirall (Specular), la part superior del cilindre es verd (Interacció amb la llum és Difusa D), el cilindre està il·luminat per la font de llum (L)

Per quadrat 1 és **LSE**: Llum (L) -> Mirall (Specular) -> Ulls (E)

quadrat 2 és: **LDE** (si pregunta pel color verd) i **LSE** (si pregunta pel punt de lluentor)

quadrat 4 és: **LDSE**

Indica quina és l'opció més adient per a completar aquest codi a l'espai o espais indicats per '_____':

```
vec4 trace(Ray ray, float mu) {
    if ( intersectScene(ray, Phit, Nhit, muHit, Kd, Kr, Kt) ) {
        Ray shadowRay = Ray(Phit, normalize(lightPos-Phit));
        bool inShadow = intersectScene(shadowRay, aux);
        if (inShadow) shadow = 0.2;
        else shadow = 1.0;
        color+= shadow*light(Nhit, -ray.dir, lightPos-Phit, Kd, vec4(1.0));
        vec3 R = reflect(ray.dir, Nhit);
        color += Kr*trace(Ray(Phit, R), mu);
        vec3 T = refract(ray.dir, Nhit, _____);
        if (length(T)>0.0) color+=Kt*trace(Ray(Phit, T), muHit);
    }
    else color+=samplePanorama(ray.dir);
    return color;
}
mu/muHit
refract(l, N, eta) on eta = índex medi origen/índex medi destí
```

20€
Por traerle

20€
Por traerle

Si eres menor de 28 años
y te haces una Cuenta NoCuenta te llevas

60€

1/6

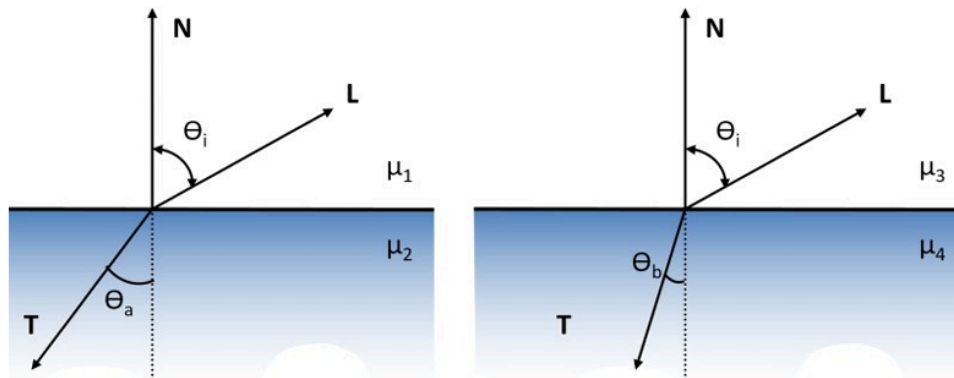
Este número es indicativo del riesgo del producto, siendo 1/6 indicativo de menor riesgo y 6/6 de mayor riesgo.

ING BANK NV se encuentra adherido al Sistema de Garantía de Depósitos Holandes con una garantía de hasta 100.000 euros por depositante. Consulta más información en ing.es

Y, si quieres más, 20€ por cada colega que te traigas* (hasta 10). Luego no digas que no avisamos.

*Promoción exclusiva para nuevos clientes menores de 28 años. Basta con hacer una compra con tarjeta de débito o realizar un Bizum al mes, durante 3 meses. Consulta bases legales en ing.es

Considerant la figura



podem afirmar que... (tria la opció correcta més completa)

- ☐ $(\mu_1 < \mu_2) \wedge (\mu_3 < \mu_4) \wedge (\mu_3/\mu_4 > \mu_1/\mu_2)$
- ☐ No vull contestar la pregunta
- ☐ $(\mu_1 < \mu_2) \wedge (\mu_3 < \mu_4) \wedge (\mu_2 > \mu_4)$
- ☐ $(\mu_1 < \mu_2) \wedge (\mu_3 < \mu_4) \wedge (\mu_1 > \mu_3)$
- ☒ $(\mu_2 > 0) \wedge (\mu_4 > 0) \wedge (\mu_1/\mu_2 > \mu_3/\mu_4)$

Llei de Snell: $n_1 \cdot \sin(\theta_1) = n_2 \cdot \sin(\theta_2)$

$$\sin(\theta_{sortida}) = \frac{\mu_{entrada}}{\mu_{sortida}} \cdot \sin(\theta_{entrada})$$

Angle incidència sembla el mateix, O_b és més petit que O_a :

$O_b < O_a \Rightarrow \sin(O_b) < \sin(O_a)$

Deduïm: $\mu_3/\mu_4 < \mu_1/\mu_2$

Opció 1 descartat, opcions 3 i 4 no podem estar segurs sense fer suposicions.

Un altre variant:

- ☐ $(\mu_1 < \mu_2) \wedge (\mu_3 < \mu_4) \wedge (\mu_2 > \mu_4)$
- ☐ $(\mu_1 < \mu_2) \wedge (\mu_3 < \mu_4) \wedge (\mu_1 > \mu_3)$
- ☐ $(\mu_1 < \mu_2) \wedge (\mu_3 < \mu_4) \wedge (\mu_1/\mu_2 < \mu_3/\mu_4)$
- ☒ $(\mu_1 < \mu_2) \wedge (\mu_3 < \mu_4) \wedge (\mu_1/\mu_2 > \mu_3/\mu_4)$ ✓

Mateixa lògica, opcions 1 i 2 fan suposicions i opció 3 és falsa.

Indica el valor que retorna aquesta expressió GLSL: `cross(vec3(3, 4, 2), vec3(2,2,4)).x`

$$A \times B = \begin{pmatrix} y_a \cdot z_b - z_a \cdot y_b \\ z_a \cdot x_b - x_a \cdot z_b \\ x_a \cdot y_b - y_a \cdot x_b \end{pmatrix}$$

$A = (x_a, y_a, z_a)$ $B = (x_b, y_b, z_b)$

Per tant retorna: $4 \cdot 4 - 2 \cdot 2 = 16 - 4 = 12$

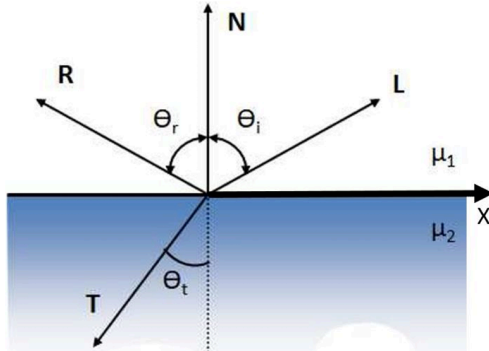


do your thing

WUOLAH

Indica el tipus de la següent expressió (en el context dels shaders del laboratori):
`mix(texCoord.s, texCoord.t, 0.5): float`

Assuming que $N = (0, 1, 0)$ i $L = (0.71, 0.70, 0.00)$, i que els dos medis tenen índexs de refracció 1.30 i 2.40, calcula (amb el signe correcte) la component X del vector unitari trasmès T.



N i L són unitaris, $\sin(\theta_i) = L_x = 0.71$

Snell: $\mu_1 \sin(\theta_i) = \mu_2 \sin(\theta_t)$

$$1.3 \cdot 0.71 = 2.4 \sin(\theta_t)$$

$\sin(\theta_t) = 0.3845$

L (sempre es superfície cap a Llum) apunta cap a la dreta (+X), per tant la llum es troba allà.
vector incident $I = -L$, per tant $T_x = -0.3845$

Indica el valor de u (enter) que fa correcte aquest codi:

```
QImage img4("file.png");
QImage T8 = img4.convertToFormat(QImage::Format_ARGB32);
glGenTextures(1, &textureId6);
glBindTexture(GL_TEXTURE_2D, textureId6);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, T8.width(), T8.height(), 0,
GL_RGBA, GL_UNSIGNED_BYTE, T8.bits());
g.glActiveTexture(GL_TEXTURE5);
g.glBindTexture(GL_TEXTURE_2D, textureId6);
program->bind();
program->setUniformValue("textureMap", u); // sampler2D
u = 5 (Hem de mirar GL_TEXTURE5)
```

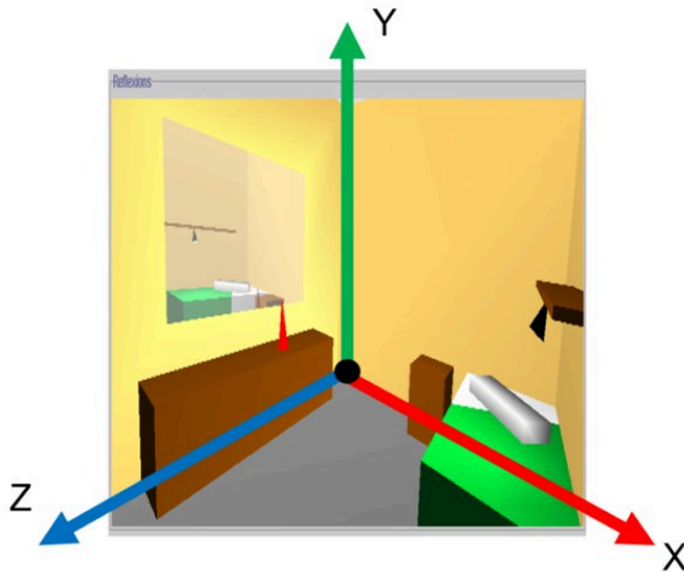
Indica la matriu que s'utilitza a la tècnica de shadow mapping per obtenir les coordenades de textura (s,t,p,q) d'un vèrtex, si el vèrtex ja es troba en 'clip space of the light camera'.

- ☒ $T(0.5) \cdot S(0.5)$
- ☐ $S(0.5) \cdot T(0.5) \cdot P \cdot V \cdot M$
- ☐ No vull contestar la pregunta
- ☐ $T(0.5) \cdot S(0.5) \cdot P$
- ☐ $M \cdot P \cdot V$

$[-1, 1] \rightarrow [0, 1]$ $\text{new} = \text{old} \cdot 0.5 + 0.5$ (primer escalem, després traslladem)

Tria l'espai de coordenades en que queda el resultat d'aplicar la transformació $\text{viewMatrix} * P$, suposant que P està en l'espai adient: eyeSpace

Considerant la figura



la matriu de reflexió per dibuixar l'escena reflectida al mirall és...

$$\begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

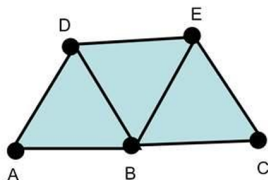
Scale(-1, 1, 1) en eix X

Tria l'espai de coordenades en que ha d'estar P per tal que la transformació $\text{projectionMatrixInverse} * P$ tingui sentit: clipSpace

Assigna a cada crida/tasca l'ordre relatiu (1,2,3,4) en que s'executa en un pipeline d'OpenGL sense GS:

1 glGenVertexArrays 2 S'escriu gl_Position 3 Backface culling 4 Rasterització

Indica un ordre adient per emetre els vèrtexs dels triangles de la figura, en un GS, usant una única primitiva: ADBEC



20€
Por traerle

20€
Por traerle

20€
Por traerle

son muchos kebabs. O mucho maquillaje.
O mucha ropa.

260€

1/6

Este número es indicativo del riesgo del producto, siendo 1/6 indicativo de menor riesgo y 6/6 de mayor riesgo.

ING BANK NV se encuentra adhiriendo al Sistema de Garantía de Depósitos Holandes con una garantía de hasta 100.000 euros por depositante. Consulta más información en ing.es

Y ahora pueden ser para ti: 60€ por abrir* tu Cuenta NoCuenta y 20€ por cada amigo que invites (hasta 10).

*Promoción exclusiva para nuevos clientes menores de 28 años. Basta con hacer una compra con tarjeta de débito o realizar un Bizum al mes, durante 3 meses. Consulta bases legales en ing.es

Montar el plan



do your thing

Indica el punt que segur que serà FORA de la piràmide de visió d'una càmera perspectiva:

(1,5,6,2) en clip FORA

(3,4,1,11) en clip

(2,0,-5,1) en eye

(-2,-4,-8,1) en eye

En clip per estar dins ha de complir: $-w \leq \{x,y,z\} \leq w$

En eye sabem que la càmera mira cap a l'eix -Z, els punts en eye que tenen una Z positiva no es veuen (darrere de la càmera).

Indica el valor que retorna aquesta expressió GLSL: $\text{mix}(9, 3, 0.7) = 4.8$

$\text{mix}(x,y,a) = x(1-a)+y*a$

Indica el valor que retorna aquesta expressió GLSL: $\text{mod}(7.9, 4) = 3.9000000000000004$

$\text{mod}(x, y) = x - y*\text{floor}(x/y)$ és el residu de la divisió

Tenim una primitiva que ocupa tot un viewport de 2048x2048 pixels. Indica quin codi ens dona un cercle blanc de radi 295 pixels centrat al viewport:

$\text{fragColor} = \text{vec4}(1-\text{step}(295, \text{distance}(\text{gl_FragCoord.xy}, \text{vec2}(1024))))$

distance retorna distancia, $\text{step}(295, \text{dist})$ si la dist < 295 retorna 0, si dist >= 295 retorna 1, 1-step inverteix el resultat.

Soposa que

P és un punt,

N és la normal unitària en el punt,

(a,b,c,d) és el pla perpendicular a N que conté P,

L és un vector unitari cap a la font de llum,

R és el vector reflectit de L,

V és un vector unitari en direcció cap a la càmera, i

Q es un punt arbitrari.

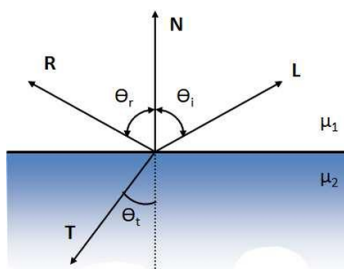
Quina interpretació té l'expressió $\text{cross}(\text{dFdx}(P), \text{dFdy}(P))$?

vector normal

Soposa que P és un punt en eye space. Una forma d'obtenir en un FS la direcció del vector normal en eye space és...

$\text{normalize}(\text{cross}(\text{dFdx}(P), \text{dFdy}(P)))$

Tenint en compte la llei de Snell, indica quina parella de valors (μ_1, μ_2) explicarien de forma aproximada la direcció del raig traspàs que s'observa a la figura:



Select one:

☒ (1.09, 1.43)

☐ (1.43, 1.09)

☐ (0.09, 1.43)

☐ (1.09, 0.43)

Ot < Oi, $\mu_1 < \mu_2$, opció 3 exagerat

WUOLAH

Amb les matrius indicades, calcula la distància al pla de retallat posterior (zfar):

$$\text{projectionMatrix} = \begin{bmatrix} 0.5 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.5 & 0.0 & 0.0 \\ 0.0 & 0.0 & -1.25 & 2.25 \\ 0.0 & 0.0 & -1.0 & 0.0 \end{bmatrix}$$
$$\text{projectionMatrixInv} = \begin{bmatrix} 2.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 2.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & -1.0 \\ 0.0 & 0.0 & 0.4444 & -0.5556 \end{bmatrix}$$

$$A = -1.25 = -(f+n)/(f-n)$$

$$\text{Les simplifiquem: near} = \text{abs}(B/(A-1))$$

$$\text{far} = \text{abs}(2.25/-0.25) = \text{abs}(-9) = 9$$

$$B = 2.25 = -2fn/(f-n)$$

$$\text{far} = \text{abs}(B/(A+1))$$

Indica quina tasca/opció pot fer que alguns fragments no segueixin processant-se:

- ☐ glClear
- ☐ glColorMask
- ☒ depth test ✓
- ☐ alpha Blending

Assigna a cada crida/tasca l'ordre relatiu en que s'executa en una pipeline d'OpenGL sense GS:

1. Call to glDrawArrays
2. Rasterization
3. fragColor is written
4. Stencil Test

Assigna a cada crida/tasca l'ordre relatiu en que s'executa en una pipeline d'OpenGL sense GS:

1. VS execution starts
2. gl_Position is written
3. Clipping to viewing frustum
4. Backface culling

Tria l'espai de coordenades en que ha d'estar P per tal que la transformació $\text{modelViewMatrix} * P$ tingui sentit: object space

En un Fragment Shader es treballa amb coordenades de textura (s,t). Dos fragments adjacents en direcció x tenen valors de $s = 0.179$ i $s = 0.794$. Quin és aproximadament el valor retornat per $dFdx(4*s)$? 2.46

$$\text{Delta } s = 0.794 - 0.179 = 0.615 \quad dFdx(4s) = 4 * dFdx(s) \quad \rightarrow \quad 4 * 0.615 = 2.46$$

Tria l'espai de coordenades en que ha d'estar P per tal que la transformació $\text{modelMatrix} * P$ tingui sentit: object space

Donat aquest codi per generar reflexions en miralls,

```
// 1. Dibuixem el mirall a l'stencil buffer
glEnable(GL_STENCIL_TEST);
glStencilFunc(GL_ALWAYS, 1, mask);
glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);
glDepthMask(GL_FALSE); glColorMask(GL_FALSE...);
dibuixar(mirall);
// 2. Dibuixem els objectes virtuals
glDepthMask(GL_TRUE); glColorMask(GL_TRUE...);
glStencilFunc(GL_EQUAL, 1, mask);
glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
glPushMatrix(); glMultMatrix(matriu simetria)
glLightfv(GL_LIGHT0, GL_POSITION, pos);
glCullFace(GL_FRONT);
dibuixar(escena);
glPopMatrix();
// 3. Dibuixem el mirall semitransparent
glDisable(GL_STENCIL_TEST);
glLightfv(GL_LIGHT0, GL_POSITION, pos);
glCullFace(GL_BACK);
dibuixar(mirall);
// 4. Dibuixem els objectes reals
dibuixar(escena);
```

un valor de la variable mask que preserva la correctesa del resultat és...

3

Volem dibuixar el reflex allà on hem marcat amb un 1. GL_EQUAL i ref és 1.

Amb mask = 0: $(1 \& 0) == (\text{stored} \& 0)$

$0 == 0$ això és sempre Cert, el reflex es dibuixa a tota la pantalla (ignorant el mirall)

Amb mask = 0xFF o 1: $(1 \& 1) == (\text{stored} \& 1)$

Si al buffer hi ha mirall (1): $1 == 1$, cert i dibuixa

Si al buffer hi ha paret (0): $1 == 0$, fals i no dibuixa

Un estimador de la il·luminació global amb molta variància es distingeix per...

- ☒ Genera imatges que requereixen noise filtering
- ☐ Suporta una gran varietat de camins de llum, especialment especulars.
- ☐ Té problemes per simular camins de llum especulars
- ☐ Suporta una gran varietat de materials (difosos i especulars)

“Genera imatges amb molt soroll” també és una resposta correcta.

20€
Por traerle

20€
Por traerle

20€
Por traerle

son muchos kebabs. O mucho maquillaje.
O mucha ropa.

260€

1/6

Este número es indicativo del riesgo del producto, siendo 1/6 indicativo de menor riesgo y 6/6 de mayor riesgo.

ING BANK NV se encuentra adhiriendo al Sistema de Garantía de Depósitos Holandeses con una garantía de hasta 100.000 euros por depositante. Consulta más información en [ing.es](https://www.ing.es)

Y ahora pueden ser para ti: 60€ por abrir* tu Cuenta NoCuenta y 20€ por cada amigo que invites (hasta 10).

Indica quina ha de ser la funció G per tal que aquest FS tingui sentit. Escriu únicament l'identificador de la funció GLSL.

```
#version 330 core
uniform mat3 normalMatrix;
in vec2 vtxCoord;
in vec3 pos; // posicio en object space
out vec4 fragColor;
void main() {
    vec3 q = pos;
    vec3 dx = dFdX(q);
    vec3 dy = dFdY(q);
    vec3 n = G(dx,dy);
    n = normalize(normalMatrix * n);
    fragColor = vec4(n.z);
}
```

G = cross

Si volem crear una piràmide de mipmapping completa a partir d'una textura de 128x128 texels, quants nivells de detall (LoD) hem de definir? $\log(128) + 1 = 8$

Indica el rang de valors de la coordenada z d'un punt visible en clip space:

- ☐ $[-\infty, 0]$
- ☐ $[0, 1]$
- ☐ $[-z, z]$
- ☒ $[-w, w]$ ✓

En el mipmap d'una textura de 1024x1024 texels, un texel del LOD 9 correspon a un grup de NxN texels del LOD 0. Indica el valor de N (enter): $2^9 = 512$

Indica el tipus de la següent expressió (en el context dels shaders del laboratori): `texture(colorMap, vtxCoord)`: vec4

Indica quina és l'aplicació de la tècnica de MipMapping: Better texture sampling

Associa cada cas amb l'etapa on seria més adient generar les coordenades de textura:

Coords de textura bàsiques per a un model 3D que no en té: **Aplicació OpenGL**

Mesh parameterization minimizing angle distortion: **Eina modelat 3D (Blender)**

Coords de textura que depenen linealment de la normal en eye space: **VS**

Coords de textura que depenen no linealment de la càmera: **FS**

*Promoción exclusiva para nuevos clientes menores de 28 años. Basta con hacer una compra con tarjeta de débito o realizar un Bizum al mes, durante 3 meses. Consulta bases legales en [ing.es](https://www.ing.es)

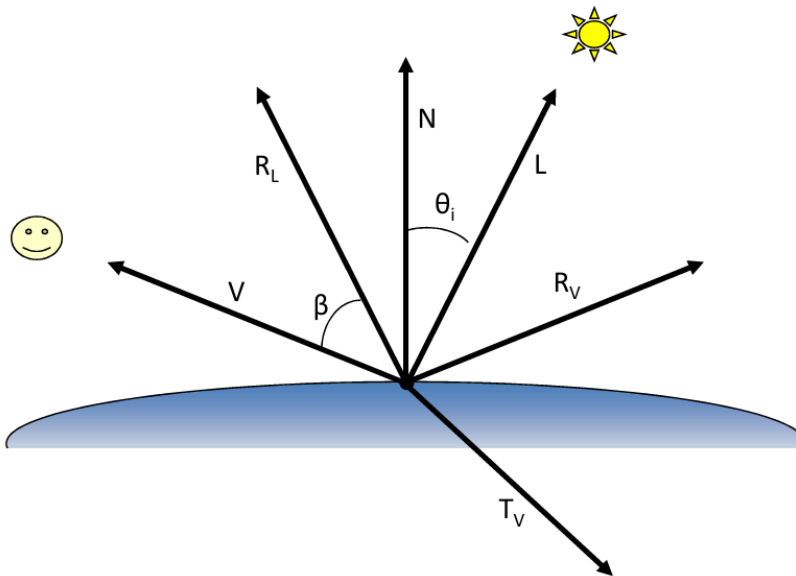
Montar el plan



do your thing

WUOLAH

La figura mostra diferents vectors unitaris que intervenen en els càlculs d'il·luminació.



Quins vectors corresponen a direccions de rajos que traçaria two-pass raytracing, durant el light pass? (tria la més completa de les correctes)

- ☐ $-L, R_V, T_V$
- ☐ $-L, R_V$
- ☒ $-L, R_L$
- ☐ R_L, R_V

Two-pass raytracing té dues passades:

1. **Eye Pass:** es llencen rajos desde l'observador.
Vector V (direcció cap a l'ull), si la superfície és reflectant o transparent, es generen R_v (reflexió de la vista) i T_v (transmissió/refracció de la vista).
2. **Light Pass:** es llencen rajos desde la font de la llum.
Vector L (raig de llum cap a la superfície), en xocar amb la superfície rebota i surt R_L (reflexió de la llum)

La figura mostra un triangle definit pels punts A, B, C.

Indica les coordenades baricèntriques (alfa, beta, gamma) del punt indicat amb un 4.

- ☐ $(1/2, 0, 1/2)$
- ☐ $(1/2, 1/2, 1/2)$
- ☒ $(1/3, 1/3, 1/3)$
- ☐ $(1/2, 1/2, 0)$

Les coordenades baricèntriques han de sumar 1 (segona opció impossible), a més si una coordenada és 0, el punt es troba sobre una de les arestes del triangle (només és possible la tercera opció).



¿Estudias en Cataluña y juegas a VALORANT?

Exámenes por la mañana, Finales de VCT EMEA por la tarde. Este Agosto disfruta del mejor VALORANT en Barcelona

Hazte con tu entrada



Siguin:

M: submatriu 3x3 de la modelMatrix

V: submatriu 3x3 de la viewMatrix,

P: submatriu 3x3 de la projectionMatrix,

la matriu que ens permet transformar normals de object space a world space es pot calcular com...

- ☐ $(MV)^{-1}$
- ☒ M^{-T}
- ☐ M^T
- ☐ $(MV)^{-T}$

Assigna a cada crida/tasca l'ordre relatiu en que s'executa en un pipeline d'OpenGL sense GS:

VS execution finishes	3	✓
Viewport transformation	4	✓
glBufferData	2	✓
glGenVertexArrays	1	✓

Indica el valor que ha de tenir la constant **S** en aquest fragment de codi:

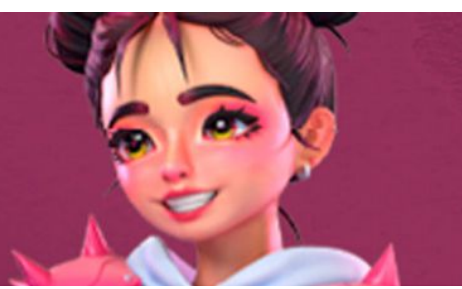
```
const unsigned int S = ____;  
GLubyte read[S];  
glReadPixels(x, y, 4, 3, GL_RGB, GL_UNSIGNED_BYTE, read);
```

S = 36

4 píxels * 3 píxels * 3 components (GL_RGB i GL_UNSIGNED_BYTE indica que cada component ocupa 1 byte) = 36 bytes

En un Vertex Shader es vol interpol·lar el color entre blau i verd segons la component **z** de la normal. Quina és l'opció correcta?

- ☐ `vec3 color = mix(0.0, 1.0, normal.z);`
- ☒ `vec3 color = mix(vec3(0.0, 0.0, 1.0), vec3(0.0, 1.0, 0.0), normal.z * 0.5 + 0.5);` ✓
- ☐ `vec3 color = mix(vec3(0.0, 0.0, 1.0), vec3(0.0, 1.0, 0.0), normal.z);`
- ☐ `vec3 color = mix(vec3(0.0, 1.0, 0.0), vec3(0.0, 0.0, 1.0), normal.z - 0.5);`



PIDE INFO Y
RESERVA TU
PLAZA

En aquest fragment de codi, la matriu M ha de ser...

```
// Pass 2. Draw the scene using the shadowmap
glClear(GL_DEPTH_BUFFER_BIT | GL_COLOR_BUFFER_BIT);
program->bind();
program->setUniformValue("lightViewMatrix", lightCamera.viewMatrix());
program->setUniformValue("lightProjectionMatrix", lightCamera.projectionMatrix());
QMatrix4x4 M = ____;
program->setUniformValue("biasMatrix", M);
program->setUniformValue("lightPos", lightCamera.getObs());
...
drawPlugin()->drawScene();
```

VS:

```
...
out vec4 vtxCoord;
void main() {
    vtxCoord = biasMatrix*lightProjectionMatrix*lightViewMatrix*vec4(vertex,1.0);
... }
```

$$\begin{bmatrix} 0.5 & 0.0 & 0.0 & 0.5 \\ 0.0 & 0.5 & 0.0 & 0.5 \\ 0.0 & 0.0 & 0.5 & 0.5 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

El Bias Matrix és una matriu que escala i trasllada per 0.5

En un VS es vol que el paràmetre t variï suaument entre 0 i 1 segons la distància d entre el ratolí i el vèrtex, de manera que t=1 quan d <= 6*radius i t=0 quan d >= 13*radius. Quina de les següents expressions és la més adequada per calcular t?

- ☐ t = clamp(d / radius, 0.0, 1.0)
- ☐ t = mix(6 * radius, 13 * radius, d)
- ☒ t = 1.0 - smoothstep(6 * radius, 13 * radius, d) ✓
- ☐ t = 1.0 - smoothstep(13 * radius, 6 * radius, d)

Si vertex és un vec3, l'operació que assigna gl_Position correctament és...

- ☐ gl_Position = projectionMatrix * modelMatrix * viewMatrix * vec4(vertex, 1.0);
- ☒ gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(vertex, 1.0); ✓
- ☐ gl_Position = projectionMatrix * viewMatrix * vec4(vertex, 1.0);
- ☐ gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(vertex, 0.0);

Donat aquest codi,

```
01 // 1. Draw mirror onto stencil buffer
02 glEnable(GL_STENCIL_TEST);
03 glStencilFunc(GL_ALWAYS, 1, 1);
04 glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);
05 glDepthMask(GL_FALSE); glColorMask(GL_FALSE...);
06 draw(mirror);
07 // 2. Draw virtual objects
08 glDepthMask(GL_TRUE); glColorMask(GL_TRUE...);
09 glStencilFunc(GL_EQUAL, 1, 1);
10 glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
11 ...
13 glCullFace(GL_FRONT);
14 draw(scene);
15 ...
16 // 3. Draw mirror
17 glDisable(GL_STENCIL_TEST);
18 ...
19 glCullFace(GL_BACK);
20 draw(mirror);
21 // 4. Draw real objects
22 draw(scene);
```

indica quina substitució de la línia 04 no alteraria la imatge resultant:

`glStencilOp(GL_REPLACE, GL_KEEP, GL_REPLACE);`

El stencil test sempre passa (`GL_ALWAYS`) i per tant el primer paràmetre de `glStencilOp` no ens importa.

Un FS rep una variable in `vec3 P` amb la posició del fragment en eye space. Per calcular el light vector `L` cal usar...

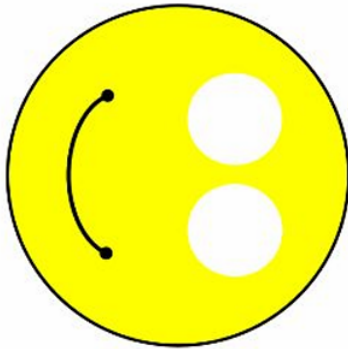
- ☐ `vec3 L = normalize((modelViewMatrix * lightPosition).xyz - P);`
- ☐ `vec3 L = lightPosition.xyz - P; L = normalize(L);`
- ☒ `vec3 L = normalize(lightPosition.xyz - modelViewMatrix * P);` 
- ☐ `vec3 L = normalize(P - lightPosition.xyz);`

La correcta és la **segona**

Tria l'espai de coordenades en que queda el resultat d'aplicar la transformació `modelViewProjectionMatrix*P`, suposant que `P` està en l'espai adient: clip space

Indica el valor positiu més petit de la constant t per tal que el FS generi aquesta sortida amb l'objecte plane i la textura smile.

```
uniform sampler2D smile;
uniform float time;
const float PI = 3.141592653589793;
const float t = ???;
void main() {
    vec2 p = vtxCoord;
    float a = 2 * PI * t;
    vec2 st = vec2( cos(a)*p.x - sin(a)*p.y, sin(a)*p.x + cos(a)*p.y);
    fragColor = texture(smile, st);
}
```

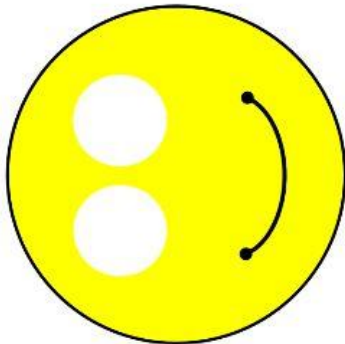


t = 0.25

Smiley rotada 90 graus en sentit horari. Per a rotar la imatge -90° , hem de rotar les coordenades 90° (en radians és $\pi/2$).

$$90 = 2\pi t \rightarrow \pi/2 = 2\pi t \rightarrow t = 1/4$$

Cas 2:



t = 0.75

Smiley rotada 90 graus en sentit antihorari. Per rotar la imatge 90° , hem de rotar les coordenades -90° (270° i per tant $3\pi/2$ radians).

$$3\pi/2 = 2\pi t \rightarrow t = 3/4$$

Suposant que modelMatrix i viewMatrix només contenen translacions i rotacions, com implementaries de forma òptima en GLSL la funció: mat3 normalMatrix(mat4 modelMatrix, mat4 viewMatrix) ?"

- ☐ return inverse(mat3(viewMatrix * modelMatrix));
- ☐ return transpose(inverse(mat3(modelMatrix * viewMatrix)));
- ☒ return mat3(viewMatrix * modelMatrix);
- ☐ return transpose(mat3(viewMatrix * modelMatrix));

Amb les matrius indicades, la posició de l'observador en model space és:

$$\text{modelMatrixInv} = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 1.0 \\ 0.0 & 1.0 & 0.0 & 1.0 \\ 0.0 & 0.0 & 1.0 & 3.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

$$\text{viewMatrixInv} = \begin{bmatrix} 0.71 & 0.3 & 0.64 & 3.0 \\ 0.0 & 0.9 & -0.43 & -2.0 \\ -0.71 & 0.3 & 0.64 & 3.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

$$\text{modelViewMatrixInv} = \begin{bmatrix} 0.71 & 0.3 & 0.64 & 4.0 \\ 0.0 & 0.9 & -0.43 & -1.0 \\ -0.71 & 0.3 & 0.64 & 6.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

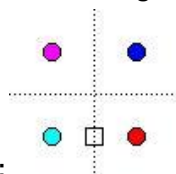
(4, -1, 6)

La posició inicial de l'observador (la càmera) es troba a l'origen. P_eye = (0, 0, 0, 1)

P_model = (modelViewMatrix)^-1 * P_eye

Al multiplicar qualsevol matriu pel vector (0, 0, 0, 1), el resultat és simplement la quarta columna d'aquesta matriu.

La figura representa un grup de 2x2 texels, amb diferents colors RGB (interior de



cada cercle):
el color RGB...

(0.5, 0.5, 0.5)

Cian: CMY(1, 0, 0) -> RGB(0, 1, 1)

Vermell: RGB(1, 0, 0)

Mitjana: (0.5, 0.5, 0.5)

. Una mostra bilinial al quadrat retornarà aproximadament

Donat el codi següent, indica el resultat de calcular una mostra de la textura al punt amb coordenades (s,t) = (0.5, 3.5).

```
// Dades de la textura (4 texels monocromàtics)
GLubyte textureData[4] = {
    170, // Texel (0,0) - lower left
    220, // Texel (1,0) - lower right
    240, // Texel (0,1) - upper left
    250  // Texel (1,1) - upper right
};

glEnable(GL_TEXTURE_2D);
GLuint textureID;
glGenTextures(1, &textureID);
glBindTexture(GL_TEXTURE_2D, textureID);
glTexImage2D(
    GL_TEXTURE_2D,
    0,           // mipmap level
    GL_LUMINANCE, // internal format
    2,           // width
    2,           // height
    0,           // border
    GL_LUMINANCE, // buffer format
    GL_UNSIGNED_BYTE, // data type
    textureData  // pointer to data
);

glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
```

220

GL_REPEAT, tenim s = 0.5 i t = 3.5 -> 0.5

La textura és de 2x2. L'espai [0,1] es divideix en 4 quadrats i els centres es trobaran a:

(0,0) -> (0.25, 0.25)

(1,0) -> (0.75, 0.25)

(0,1) -> (0.25, 0.75)

(1,1) -> (0.75, 0.75)

(s,t) = (0.5, 0.5) queda just al mig, resultat = (170+220+240+250)/4 = 220

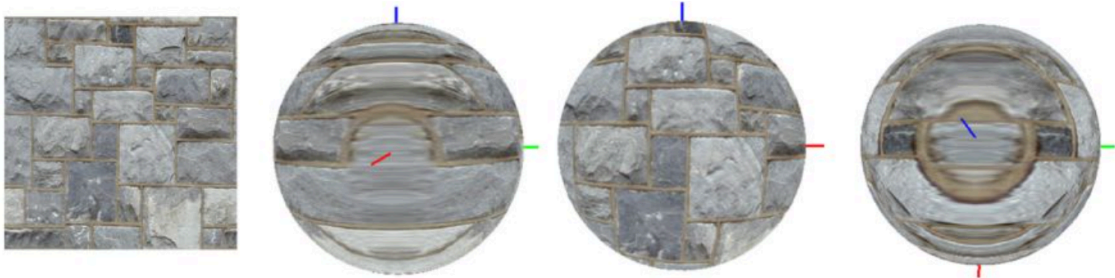
Assigna a cada crida/tasca l'ordre relatiu (1,2,3,4) en que s'executa en un pipeline d'OpenGL sense GS:

glGenVertexArrays -> Call to glDrawArrays -> Depth test -> A new color is written to the color buffer

Com a part d'un procés de generació de coordenades de textura, volem projectar un model centrat en el punt C sobre una esfera unitària. El mètode més adient és...

- ☐ `vec3 P = distance(vertex.xyz, C);`
- ☐ `vec3 P = (modelViewProjectionMatrix * vertex).xyz - C;`
- ☒ `vec3 P = vertex.xyz - C; P = normalize(P);` ✓
- ☐ `vec3 P = (modelViewProjectionMatrix * vertex).xyz;`

Indica els plans S, T que calen per texturar una esfera de radi R=2 centrada a l'origen, com indica la figura (es mostra la textura, i el resultat des dels eixos X, -Y i Z).



- ☐ `vec4 S = vec4(0, 2, 0, 0); vec4 T = vec4(2,0,0, 0);`
- ☐ `vec4 S = vec4(0.25, 0, 0, 0.5); vec4 T = vec4(0,0,0.25, 0.5);`
- ☐ `vec4 S = vec4(0.5, 0, 0, 0.5); vec4 T = vec4(0,0,0.5, 0.5);`
- ☐ `vec4 S = vec4(0, 2, 0, 0.5); vec4 T = vec4(2,0,0, 0.5);`

La segona és correcta.

Enunciat diu R = 2, això vol dir que, centrada a l'origen, les coordenades van de -2 a +2, la textura va de 0 a 1, necessitem un factor de escala $1/4 = 0.25$, descartem. També volem que el centre de l'esfera (0,0,0) coincideixi amb el centre de la textura (0.5, 0.5):

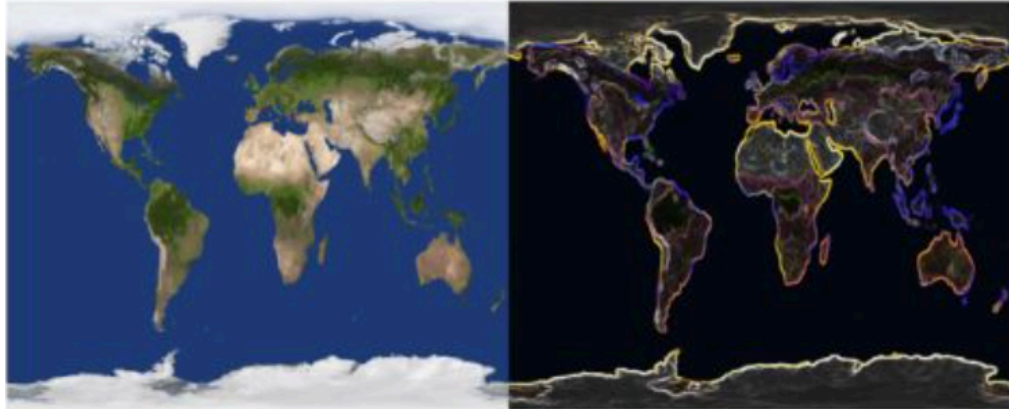
$$S = \text{scale} * x + \text{offset} \quad \rightarrow \quad 0.5 = 0.25 * 0 + \text{offset} \quad \rightarrow \quad \text{offset} = 0.5$$

Estem implementant la funció: `float specularFactor(vec3 R, vec3 V, float shininess)`
Suposa que R i V arriben en els espais indicats a continuació. Quina seria la implementació més correcta per calcular el factor especular? En cas de que n'hi hagi més d'una opció correcta, tria la que calcula el producte escalar amb els vectors en eye space. Cas: R en world space; V en object space

- ☒ `return pow(dot(mat3(viewMatrix)*R, mat3(modelViewMatrix)*V), shininess)` ✓
- ☐ `return pow(dot(mat3(viewMatrix)*R, V), shininess)`
- ☐ `return pow(dot(mat3(viewMatrix)*R, mat3(modelMatrixInverse)*V), shininess)`
- ☐ `return pow(dot(mat3(modelMatrixInverse)*R, mat3(viewMatrix)*V), shininess)`



Indica com calcular la coordenada s per texturar una esfera unitària amb una projecció equirectangular, usant únicament el mapa de la dreta. Suposa que $\theta = \text{atan}(\text{vertex.x}, \text{vertex.z}) + \pi$, amb θ en $[0, 2\pi]$.



- ☐ $s = \theta / (2\pi) + 2$
- ☐ $s = 2 * \theta / (2\pi)$
- ☐ $s = 2 * \theta / (2\pi) + 0.5$
- ☒ $s = \theta / (4\pi) + 0.5$ ✓

La textura va de 0 a 1, el mapa de l'esquerra va de 0 a 0.5, el de la dreta va de 0.5 a 1.
Volem que quan $\theta = 0$, la coordenada s sigui inici del mapa dret (0.5) i quan $\theta = 2\pi$, la coordenada s sigui el final del mapa dret (1)
Scale = rang destí/rang origen = $0.5/(2\pi) = 1/(4\pi)$
Offset = inici és a 0.5
 $S = \theta/(4\pi) + 0.5$



Volem rotar un punt 90 graus al voltant d'un eix paral·lel a l'eix Y que passa pel punt (1.00, 0.00, 7.00). Indica la matriu de transformació que cal aplicar al punt per obtenir el punt rotat:

$$\begin{bmatrix} 0.0 & 0.0 & 1.0 & -6.0 \\ 0.0 & 1.0 & 0.0 & 0.0 \\ -1.0 & 0.0 & 0.0 & 8.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix} \quad \checkmark$$

Hem de traslladar a l'origen, rotar i destralladar.

$$T = \begin{pmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -7 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad T^{-1} = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 7 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_y(90) = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M = T^{-1} * R * T$$

Transformacions útils:

Origen	->	Destí	Matriu
object space		model space	I
object space		world space	M
object space		eye space	V*M
object space		clip space	P*V*M
world space		eye space	V
world space		clip space	P*V
eye space		clip space	P
clip space		eye space	P^-1
clip space		world space	V^-1*P^-1
eye space		world space	V^-1
clip space		object space	M^-1*V^-1*P^-1

(ModelViewProjectionMatrixInverse)

Pipeline:

Object -> multiplicat per modelMatrix -> World -> multiplicat per ViewMatrix
-> Eye -> multiplicat per ProjectionMatrix -> Clip -> dividir per W
-> NDC -> escalat a píxels -> Window/Screen

Matrius:

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Translació

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Escalat

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotació X

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotació Y

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotació Z

$$P_{persp} = \begin{bmatrix} \frac{1}{ar \cdot t} & 0 & 0 & 0 \\ 0 & \frac{1}{t} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Projecció

fov, ar (ample/alt), n (near), f (far), t = tan(fov/2)